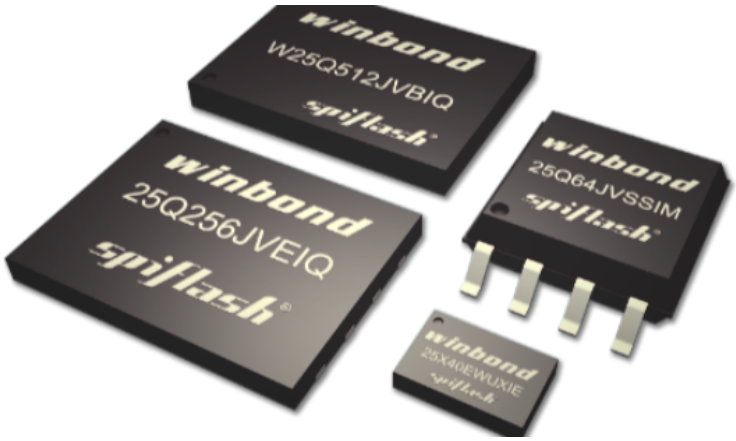


💡Unlocking the Power of W25Q Flash Memory: A Deep Dive into Firmware Development 💡

Subhrajit Majumder
Post Graduate Embedded Developer (Research and Development) | Embedded Software | Embedded Hardware |...

September 1, 2024

In the world of embedded systems , firmware development plays a crucial role in enabling devices to operate effectively and stably. One such device is the W25Q flash memory, widely used in various industries. In this article, I will delve into the firmware development for the W25Q series of flash memory, a popular choice due to its high performance and reliability.

Understanding W25Q Flash Memory

The W25Q series, manufactured by Winbond, is a family of Serial Peripheral Interface (SPI) flash memory devices. These memory chips are known for their high-speed performance, small form factor, and cost-effectiveness, making them ideal for applications ranging from consumer electronics to industrial automation. It can provide more than 20-year data retention.

Key features of W25Q flash memory include:

- **High Density**: Available in capacities from 512Kbit to 512Mbit.
- **Wide Voltage Range**: Operates between 2.7V to 3.6V.
- **Fast Read/Write Speeds**: Up to 104MHz clock rate with support for Dual and Quad I/O SPI.
- **Low Power Consumption**: Ideal for battery-operated devices.

Hardware Connection

In this demonstration, we will explore the integration of the STM32 microcontroller with the W25Q Flash memory, highlighting the benefits of using this combination for efficient data storage and retrieval. The STM32 microcontroller, known for its high-performance capabilities and low power consumption, is paired with the W25Q Flash memory, a high-capacity and reliable storage solution.

Hardware Components:

- STM32 Microcontroller
- W25Q Flash Memory Module

The W25Q NOR Flash memories can use the SPI in Single/Dual/Quad mode. For the simplicity of the operation, we will stick to the Single SPI mode, where we would need 4 pins to connect the Flash with the controller.

GPIO Connection:

- W25Q CS Pin --> connect with STM32 Chip Select Pin.
- W25Q Clock Pin --> connect with STM32 SPI Clock Pin.
- W25Q DI Pin --> connect with STM32 SPI MOSI Pin.
- W25Q DO Pin --> connect with STM32 SPI MISO Pin.
- W25Q Vcc Pin --> connect with STM32 5v or 3v3 Pin.
- W25Q GND Pin --> connect with STM32 GND Pin.

STM32CubeIDE Configuration

We will enable the SPI in **Full Duplex master mode**. The configuration is shown below.

2.3. SPI1

Mode: Full-Duplex Master

2.3.1. Parameter Settings:

Basic Parameters:	
Frame Format	Motorola
Data Size	8 Bits
First Bit	MSB First
Clock Parameters:	
Prescaler (for Baud Rate)	**16 ***
Baud Rate	**1.5625 MBits/s ***
Clock Polarity (CPOL)	Low
Clock Phase (CPHA)	1 Edge
Advanced Parameters:	
CRC Calculation	Disabled
NSS Signal Type	Software

SPI Configuration using STM32CubeIDE.

The Data width is set to **8 bits** and the data should be transferred as the **MSB first**. The prescaler is set such that the Baud Rate is around **1.5 Mbits/sec**. We can set Baud Rate upto **3Mbits/sec**.

According to the datasheet of the W25Q Flash, the SPI Mode 0 and Mode 3 are supported. Below is the image from the datasheet.

6.1.1 Standard SPI Instructions

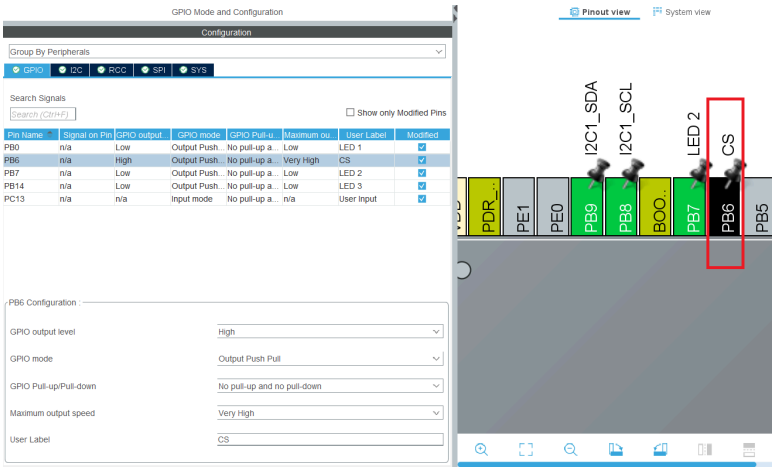
The W25Q32FV is accessed through an SPI compatible bus consisting of four signals: Serial Clock (CLK), Chip Select (/CS), Serial Data Input (DI) and Serial Data Output (DO). Standard SPI instructions use the DI input pin to serially write instructions, addresses or data to the device on the rising edge of CLK. The DO output pin is used to read data or status from the device on the falling edge of CLK.

SPI bus operation Mode 0 (0,0) and 3 (1,1) are supported. The primary difference between Mode 0 and Mode 3 concerns the normal state of the CLK signal when the SPI bus master is in standby and data is not being transferred to the Serial Flash. For Mode 0, the CLK signal is normally low on the falling and rising edges of /CS. For Mode 3, the CLK signal is normally high on the falling and rising edges of /CS.

W25Q supported SPI modes.

In the SPI configuration, we keep the Clock Polarity **(CPOL) low** and the Clock Phase **(CPHA) to 1 edge**. Here 1 edge means that the data will be sampled on the **first edge of the clock**. And when the CPOL is Low, the first edge is the **rising edge**. Basically we are using the **SPI Mode 0**.

In Full duplex Mode, SPI uses 3 pins, **MOSI**, **MISO** and **CLK**. We need to set one more pin as output so to be used as the Chip Select (**CS**) Pin.



SPI Chip Select Pin Configuration using STM32CubeIDE.

The Pin **PB6** is set as the CS pin. The initial output level is set to **HIGH**. This is because the pin needs to be pulled low in order to select the slave device, so we keep it high initially to make sure the slave device isn't selected. The Output speed is set to **Very High** because we might need to select and unselect the slave at higher rate.

Firmware for W25Q Flash Memory

First we need to define some fucntions on the main.c file.

I am defining a delay function in case we need to use the delay in the code. The implementation of the delay function can be changed based on what MCU you are using.

```
void W25Q_Delay(uint32_t time){
	HAL_Delay(time);
}
```

The chip select pin needs to be pulled High and Low whenever we want to read or write the data to the device. So we define the chip select functions separately.

```
void csLOW(void){
	HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6,
GPIO_PIN_RESET);
}

void csHIGH(void){
	HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6,
GPIO_PIN_SET);
}
```

The pin PB6 is connected to the CS pin of the device. The **csLOW()** function will simply pull the pin PB6 Low and **csHIGH()** will pull it High.

I will also define the SPI Read and Write functions. This will make it easier to modify the code for different MCUs without having to modify a lot in the code.

```
void SPI_Write(uint8_t *data, uint8_t len){
	HAL_SPI_Transmit(&hspi1, data, len, 2000);
}

void SPI_Read(uint8_t *data, uint8_t len){
	HAL_SPI_Receive(&hspi1, data, len, 2000);
}
```

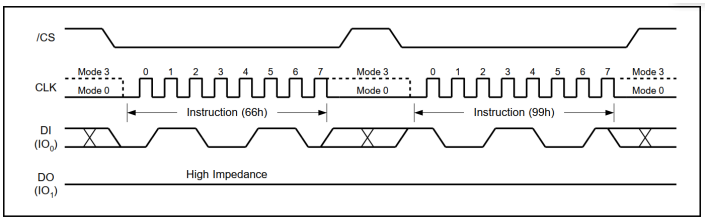
The parameter for the **SPI_Write** function is pointer to the data to be transmitted and the **length** of the data. It then passes the same parameters to the **HAL_SPI_Transmit** function.

Similarly, the **SPI_Read** function reads the **"len"** number of bytes, and store them in the **array** whose address is passed to the function.

Resetting the Chip

Before starting a ne session with the chip we should reset it. This would terminate any on going operation and the device will return to its default power-on state. It will lose all the current volatile settings, such as Volatile Status Register bits, Write Enable Latch (WEL) status, Program/Erase Suspend status, Read parameter setting (P7-P0) and Wrap Bit setting (W6-W4).

To avoid the accidental Reset, a Reset command is made of 2 instructions. these Instructions, "**Enable Reset (66h)**" and "**Reset (99h)**" must be issues in a sequence. Once the Reset command is accepted by the device, the device will take approximately 30us to reset. During this period, no command will be accepted.



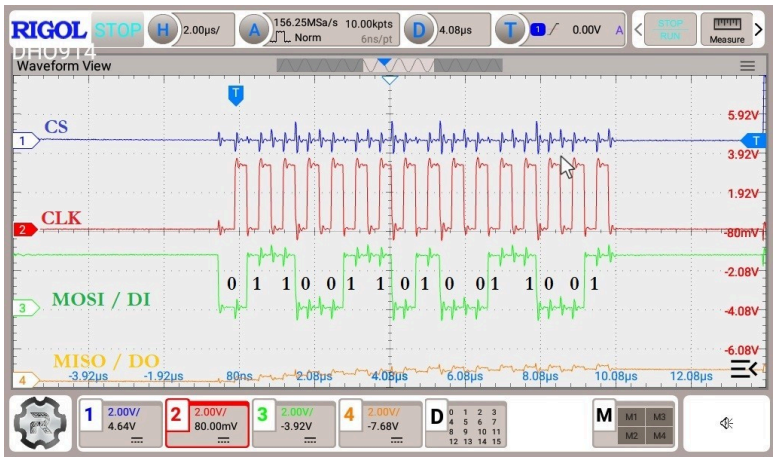
SPI instruction for Enable Reset (66h) and Reset Device (99h).

I will write a separate function to reset the chip. The code is shown below

```
void W25Q_Reset(void){
	uint8_t tData[2];
```

```
        tData[0] = 0x66; // Reset Enable; Binary:  0110
0110
        tData[1] = 0x99; // Reset Trigger; Binary: 1001
1001
        csLOW();
        SPI_Write(tData, 2);
        csHIGH();
        W25Q_Delay(100);
}
```

- Since we need to send 2 instructions, we define an array of 2 bytes.
- Then store the instructions in sequence with Enable Reset followed by the Reset Instruction.
- Pull the CS Low to select the device, send the 2 bytes data using SPI_Write and Pull the CS High to unselect the device.
- Provide 100ms delay so that the things can settle after the reset.



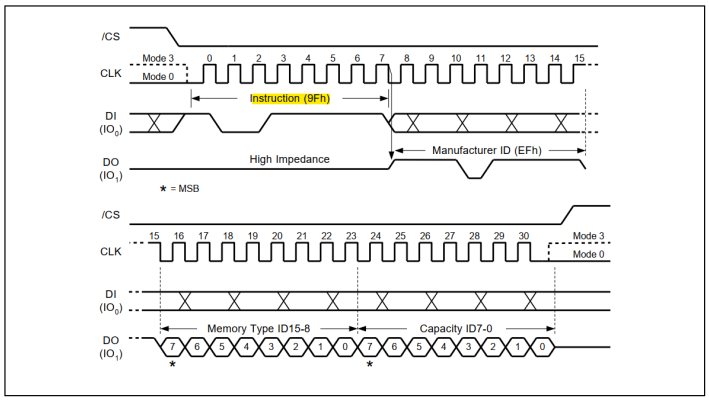
Waveform captured as respect to the W25Q_Reset() function.

Reading ID

I will also Read the ID of the device. The W25Q series devices are equipped with different IDs like manufacturer ID, Device ID, Unique ID and JEDEC ID. We will read the **JEDEC ID** as it give a lot of useful information about the device.

The instruction 9Fh can be used to read the JEDEC ID.

The JEDEC assigned Manufacturer ID byte for Winbond (EFh) and two Device ID bytes, Memory Type (ID15-ID8) and Capacity (ID7-ID0) are then shifted out on the falling edge of CLK with most significant bit (MSB) first.



Read JEDEC ID Instruction.

The manufacturer ID (EFh) remains same across all the winbond flash memories. The Memory Type ID and the Capacity ID changes based on what device you are using.

I have the W25Q128JV chip, which is 16 Megabits in size. The ID for this chip is shown below.

8.1.1 Manufacturer and Device Identification

MANUFACTURER ID	**(MF7 - MF0)**	
Winbond Serial Flash	EFh	
Device ID	**(ID7 - ID0)**	**(ID15 - ID0)**
Instruction	**ABh, 90h, 92h, 94h**	**9Fh**
W25Q128JV-IQ/JQ	17h	4018h
W25Q128JV-IM*/JM*	17h	7018h

Note: For DTR, QPI supporting, please refer to W25Q128JV DTR datasheet.

JEDEC ID for W25Q128JV chip.

The Manufacturer ID is 0xEF and the 2 device ID bytes are going to be 0x4018. Below is the function to read the ID.

```
uint32_t ID = 0;

uint32_t W25Q_ReadID(void){
        uint8_t tData = 0x9F; // Binary: 1001 1111
        uint8_t rData[3];
        csLOW();
        SPI_Write(&tData, 1);
        SPI_Read(rData, 3);
        csHIGH();
        return ((rData[0] << 16) | (rData[1] << 8) |
rData[2]);
}

int main ()
{

  W25Q_Reset();
  ID = W25Q_ReadID();
```

```
}
```

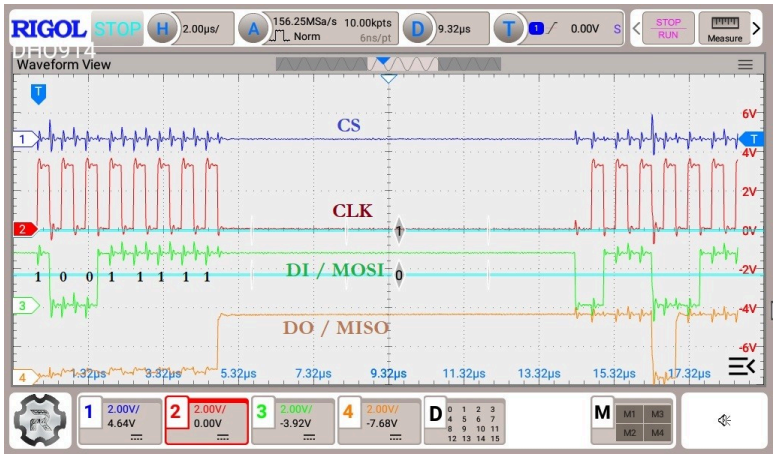
We define a variable and store the instruction (**0x9F**) to Read the ID. The array **rData** is used to store the 3 received bytes.

Then send the instruction using **SPI_Write()** function and read 3 bytes using the **SPI_Read()** function.

The rData[0] stores the Manufacturer ID, rData[1] stores the Memory ID and the rData[2] stores the Capacity ID. We combine the 3 bytes and make a final 24 bit ID in the format **MFN ID : MEM ID : CAPACITY ID**. Finally return the 24 bit ID.

Expression	Type	Value
(x)= ID	uint32_t	0xef4018

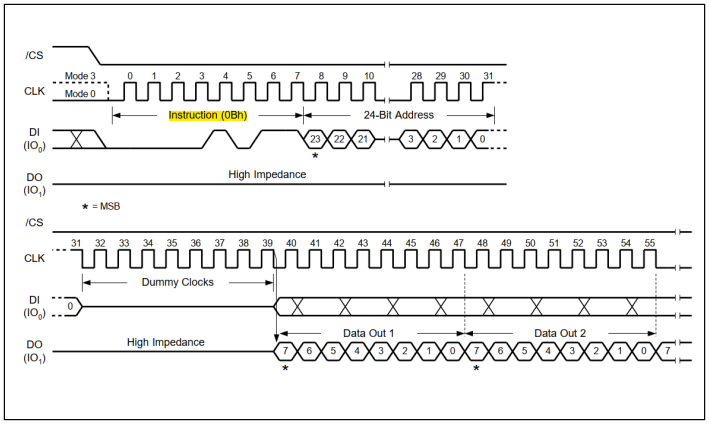
Read JEDEC ID using STM32CubeIDE debugger.



Waveform captured as respect to the W25Q_ReadID() function.

FastRead Command

The Fast Read instruction can operate at the highest possible frequency.The Fast Read is accomplished by adding eight “dummy” clocks after the 24/32-bit address. The dummy clocks allow the devices internal circuits additional time for setting up the initial address.



SPI instruction for FastRead Command.

I will write the function for **FastRead**, with the addition of dummy clocks in the command. Below the code shows the Fast Read function.

```
uint8_t RxData[4096];
uint8_t TxData[30];

void W25Q_FastRead(uint32_t startpage, uint8_t offset,
uint32_t length, uint8_t *rData){
        uint8_t tData[6];
        uint32_t memAddr = (startpage * 256) + offset;
        tData[0] = 0x0B;
        tData[1] = (memAddr >> 16) & 0xFF;
        tData[2] = (memAddr >> 8) & 0xFF;
        tData[3] = (memAddr) & 0xFF;
        tData[4] = 0;
        csLOW();
        SPI_Write(tData, 5);
        SPI_Read(rData, length);
        csHIGH();
}

int main(){
       W25Q_FastRead(0, 0, 512, RxData);
                                    // Read 512 bytes (2
Pages) from the start of Page 0

      W25Q_FastRead(16, 0, 512, RxData);
                                   // Read 512 bytes (2
Pages) from the start of Page 16

     W25Q_FastRead(0, 0, 4608, RxData);
                                  // Read 4608 bytes (18
Pages) from the start of Page 0
}
```

Result

I have stored the data "**Hello world**" at 2 different locations in the memory. Both locations are in different sectors, **sector 0 (page0 to page15)** and **sector 1 (page16 to page31)**.

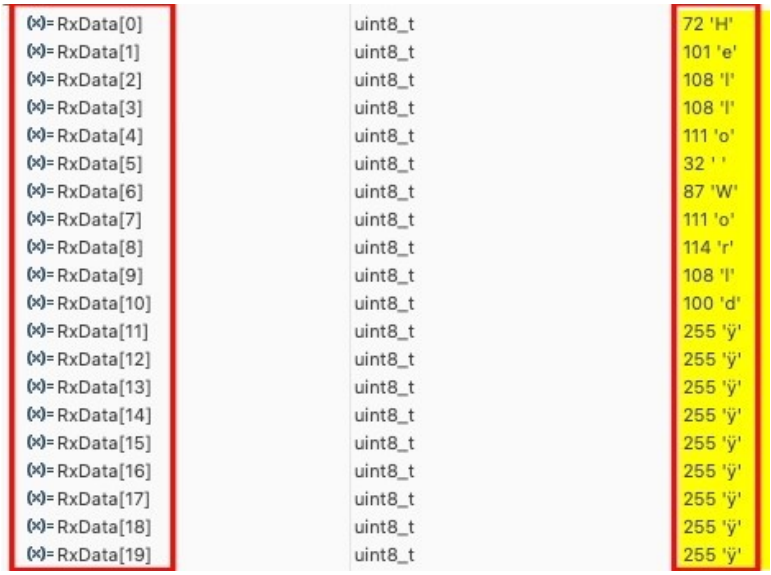
I intentionally chose the different sectors, to demonstrate that we can perform continuous read even between the sectors. The image below shows how the data is placed and what is the memory address for the corresponding byte.

	PAGE 0				PAGE 1														
Offset	0	1	2	255	0	1	2	--	--	85	86	87	88	89	90	91	92	93	94	95
Address	0	1	2	255	256	257	258	--	--	341	342	343	344	345	346	347	348	349	350	351
DATA	0xFF	0xFF	0xFF	0xFF	0xFF	0xFF	0xFF	0xFF	0xFF	H	e	l	l	o		w	o	r	l	d

	PAGE 16				PAGE 17														
Offset	0	1	2	255	0	1	2	--	--	10	11	12	13	14	15	16	17	18	19	20
Address	4096	4097	4098	4052	4053	4056	4057	--	--	4362	4363	4363	4364	4365	4366	4367	4368	4369	4370	4371
DATA	0xFF	0xFF	0xFF	0xFF	0xFF	0xFF	0xFF	0xFF	0xFF	H	e	l	l	o		w	o	r	l	d

W25Q Main memory map.

Let's see the output of the first statement, i.e. **W25Q_FastRead(0, 0, 512, RxData);** // Read 512 bytes (2 Pages) from the start of Page 0



(x)= RxData[0]	uint8_t	72 'H'
(x)= RxData[1]	uint8_t	101 'e'
(x)= RxData[2]	uint8_t	108 'l'
(x)= RxData[3]	uint8_t	108 'l'
(x)= RxData[4]	uint8_t	111 'o'
(x)= RxData[5]	uint8_t	32 ' '
(x)= RxData[6]	uint8_t	87 'W'
(x)= RxData[7]	uint8_t	111 'o'
(x)= RxData[8]	uint8_t	114 'r'
(x)= RxData[9]	uint8_t	108 'l'
(x)= RxData[10]	uint8_t	100 'd'
(x)= RxData[11]	uint8_t	255 'ÿ'
(x)= RxData[12]	uint8_t	255 'ÿ'
(x)= RxData[13]	uint8_t	255 'ÿ'
(x)= RxData[14]	uint8_t	255 'ÿ'
(x)= RxData[15]	uint8_t	255 'ÿ'
(x)= RxData[16]	uint8_t	255 'ÿ'
(x)= RxData[17]	uint8_t	255 'ÿ'
(x)= RxData[18]	uint8_t	255 'ÿ'
(x)= RxData[19]	uint8_t	255 'ÿ'

Capture data using W25Q_FastRead(0, 0, 512, RxData) function using STM32CubeIDE debugger.

The image above shows that we received the data, "Hello world", at the beginning of the RxData buffer. This is as expected because we read from the offset of 85. This is where the data is actually stored in the memory.

The next statement reads the data from the start of the page 16. This time we should get data in the RxData buffer at an offset of 266 (1×256 + 10). This is the actual position of the data from the start of the page 16 as shown in the figure main memory map.

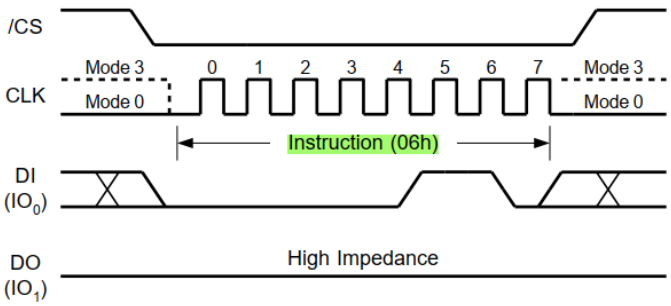
(x)= RxData[0]	uint8_t	72 'H'
(x)= RxData[1]	uint8_t	101 'e'
(x)= RxData[2]	uint8_t	108 'l'
(x)= RxData[3]	uint8_t	108 'l'
(x)= RxData[4]	uint8_t	111 'o'
(x)= RxData[5]	uint8_t	32 ' '
(x)= RxData[6]	uint8_t	87 'W'
(x)= RxData[7]	uint8_t	111 'o'
(x)= RxData[8]	uint8_t	114 'r'
(x)= RxData[9]	uint8_t	108 'l'
(x)= RxData[10]	uint8_t	100 'd'
(x)= RxData[11]	uint8_t	255 'ÿ'
(x)= RxData[12]	uint8_t	255 'ÿ'
(x)= RxData[13]	uint8_t	255 'ÿ'
(x)= RxData[14]	uint8_t	255 'ÿ'
(x)= RxData[15]	uint8_t	255 'ÿ'
(x)= RxData[16]	uint8_t	255 'ÿ'
(x)= RxData[17]	uint8_t	255 'ÿ'
(x)= RxData[18]	uint8_t	255 'ÿ'
(x)= RxData[19]	uint8_t	255 'ÿ'

Capture data using W25Q_FastRead(16, 0, 512, RxData) function using STM32CubeIDE debugger.

Waveform captured as respect to the W25Q_FastRead() function.

Write Enable & Disable

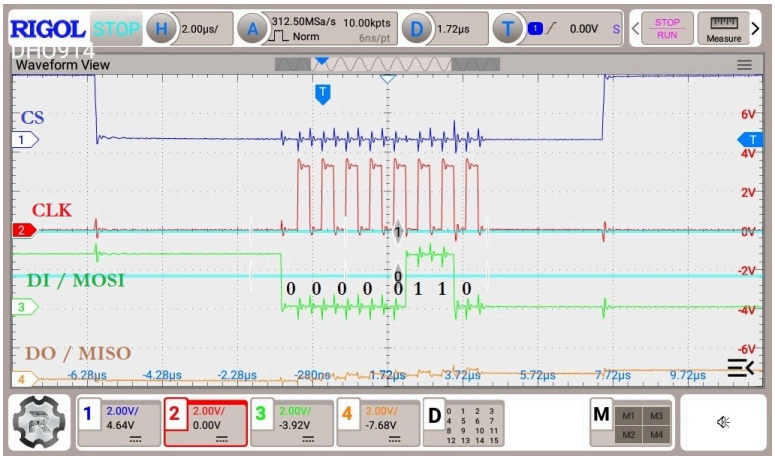
The write enable instruction must be executed prior to every Page Program, Sector Erase, Block Erase, Chip Erase, Write Status Register and Erase/Program Security Registers instruction. The instruction is shown below.



Write Enable Instruction for SPI Mode.

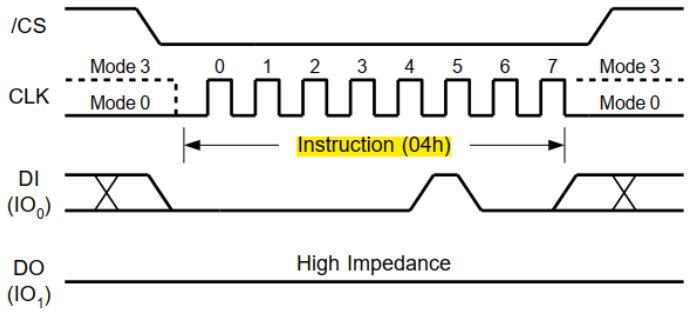
```
void enable_write (void)
{
        uint8_t tData = 0x06;  // enable Write
        csLOW();  // pull the CS LOW
        SPI_Write(&tData, 1);
        csHIGH();  // pull the HIGH
        W25Q_Delay (5);  // Write cycle delay (5ms)
}
```

We simply send the instruction (0x06) to the device. This enables the write and we can perform the page write or erasing operations after this.



Waveform captured as respect to the enable_write() function.

Just like write enable, there is an instruction to disable the write also. Below is the code for the same.

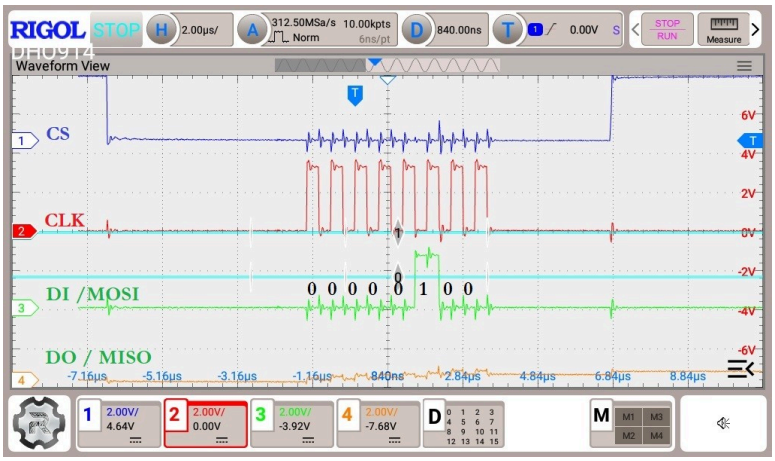


Write Disable Instruction for SPI Mode.

```
void disable_write (void)
{
        uint8_t tData = 0x04;  // disable Write
        csLOW();  // pull the CS LOW
```

```
        SPI_Write(&tData, 1);
        csHIGH();  // pull the HIGH
        W25Q_Delay (5);  // Write cycle delay (5ms)
}
```

The instruction is performed in the similar way as the write enable instruction, except the instruction (0x04) is different. Note that the write is automatically disabled after Power-up and upon completion of the Write Status Register, Erase/Program Security Registers, Page Program, Sector Erase, Block Erase, Chip Erase and Reset instructions. I am just including it so as to perform the safe operations.



Waveform captured as respect to the disable_write() function.

Erase Sectors

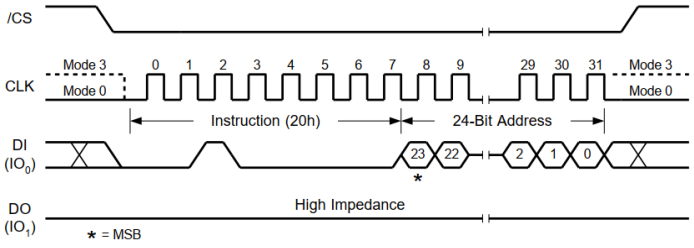
Erasing a sector is important before we learn about writing data into the flash memory. According to the datasheet of the module, we can only write the data at some location, if it has been erased (current data is 0xFF). Also a sector is the smallest memory that we can erase.

> ***That means we can not erase a particular memory byte, or even a page. Although we can erase a complete sector (16 pages), or a 32 KB memory block (8 sectors), or a 64 KB memory block (16 sectors), or even a complete chip. But the higher the memory block we erase, the more time consuming the process is.***

For getting started, erasing a sector is good enough and it only takes 400ms to do so. This information is provided in the datasheet of the device.

DESCRIPTION	SYMBOL	ALT	SPEC			UNIT
			MIN	TYP	MAX	
Write Protect Setup Time Before /CS Low	t_{WHSL}(3)		20			ns
Write Protect Hold Time After /CS High	t_{SHWL}(3)		100			ns
/CS High to Power-down Mode	t_{DP}(2)				3	µs
/CS High to Standby Mode without ID Read	t_{RES1}(2)				3	µs
/CS High to Standby Mode with ID Read	t_{RES2}(2)				1.8	µs
/CS High to next Instruction after Suspend	t_{SUS}(2)				20	µs
/CS High to next Instruction after Reset	t_{RST}(2)				30	µs
/RESET pin Low period to reset the device	t_{RESET}(2)		1(4)			µs
Write Status Register Time	t_W			10	15	ms
Page Program Time	t_{PP}			0.4	3	ms
Sector Erase Time (4KB)	t_{SE}			45	400	ms
Block Erase Time (32KB)	t_{BE1}			120	1,600	ms
Block Erase Time (64KB)	t_{BE2}			150	2,000	ms
Chip Erase Time	t_{CE}			40	200	s

Sector Erase MAX time as per datasheet.



Sector Erase Instruction for SPI Mode.

The erase sector instruction (**0x20**) is sent along with the memory address, whose sector should be erased. We send the instruction (**0x21**) for the same, but for the 32bit memory addresses (size>=256Mb). The Sector Erase instruction sets all memory within a specified sector (4K-bytes) to the erased state of all 1s (**0xFF**). A Write Enable instruction must be executed before the device will accept the Sector Erase Instruction.

```
void W25Q_Erase_Sector (uint16_t numsector)
{
        uint8_t tData[6];
        uint32_t memAddr = numsector*16*256;   // Each
sector contains 16 pages * 256 bytes

        write_enable();

        if (numBLOCK<512)   // Chip Size<256Mb
        {
          tData[0] = 0x20;  // Erase sector
          tData[1] = (memAddr>>16)&0xFF;  // MSB of the
memory Address
          tData[2] = (memAddr>>8)&0xFF;
          tData[3] = (memAddr)&0xFF; // LSB of the memory
Address

          csLOW();
          SPI_Write(tData, 4);
```

```
          csHIGH();
        }
        else  // we use 32bit memory address for chips >=
256Mb
        {
          tData[0] = 0x21;  // ERASE Sector with 32bit
address
          tData[1] = (memAddr>>24)&0xFF;
          Data[2] = (memAddr>>16)&0xFF;
          tData[3] = (memAddr>>8)&0xFF;
          tData[4] = memAddr&0xFF;

          csLOW();  // pull the CS LOW
          SPI_Write(tData, 5);
          csHIGH();  // pull the HIGH
        }

        W25Q_Delay(450);  // 450ms delay for sector erase

        write_disable();

}

int main(){
      W25Q_Erase_Sector(0);
}
```

The parameter of the function is the number of sector that we want to erase.

- We first calculate the memory address using the sector number. Each sector contains 16 pages and each page contains 256 bytes.
- The memory address could be 24bit / 32bit depending on the memory size of the device.
- We use different commands for both address sets. 0x20 for the 24 bit memory addresses and 0x21 for the devices with 32bit memory addresses.
- Before sending the erase command, we enable the write.
- To erase the sector we send the instruction followed by the memory address.
- The sector erase time is 400ms, so we wait for the erase to complete.
- At last disable the write.

We use the same code from the previous tutorial, where we read the data from 2 different sectors. We erase the sector 0 and sector 1, and again read the data.

If the sectors gets erased, we should receive the data 0xFF instead of what was stored in those positions.

Expression	Type	Value
[0..99]		[4096]
RxData[0]	uint8_t	255 'ÿ'
RxData[1]	uint8_t	255 'ÿ'
RxData[2]	uint8_t	255 'ÿ'
RxData[3]	uint8_t	255 'ÿ'
RxData[4]	uint8_t	255 'ÿ'
RxData[5]	uint8_t	255 'ÿ'
RxData[6]	uint8_t	255 'ÿ'
RxData[7]	uint8_t	255 'ÿ'
RxData[8]	uint8_t	255 'ÿ'
RxData[9]	uint8_t	255 'ÿ'
RxData[10]	uint8_t	255 'ÿ'
RxData[11]	uint8_t	255 'ÿ'
RxData[12]	uint8_t	255 'ÿ'
RxData[13]	uint8_t	255 'ÿ'
RxData[14]	uint8_t	255 'ÿ'
RxData[15]	uint8_t	255 'ÿ'
RxData[16]	uint8_t	255 'ÿ'
RxData[17]	uint8_t	255 'ÿ'
RxData[18]	uint8_t	255 'ÿ'
RxData[19]	uint8_t	255 'ÿ'
RxData[20]	uint8_t	255 'ÿ'
RxData[21]	uint8_t	255 'ÿ'
RxData[22]	uint8_t	255 'ÿ'
RxData[23]	uint8_t	255 'ÿ'
RxData[24]	uint8_t	255 'ÿ'
RxData[25]	uint8_t	255 'ÿ'
RxData[26]	uint8_t	255 'ÿ'
RxData[27]	uint8_t	255 'ÿ'
RxData[28]	uint8_t	255 'ÿ'

Receive the data 0xFF (Using STM32CubeIDE debugger).

Waveform captured as respect to the W25Q_Erase_Sector() function.

Page Write Function

I have already covered the functions to **enable the write** and to **erase the sectors**, so I will only focus on the page program part.

```
uint8_t RxData[4096];
uint8_t TxData[30];
```

```
void W25Q_Write(uint32_t page, uint16_t offset, uint32_t
length, uint8_t *data){
        uint8_t tData[256];
        uint32_t indx = 4;
        uint16_t startsector = page / 16;
        uint16_t endsector = (page + ((offset + length -1)
/ 256))/16;
        uint32_t numSectors = endsector - startsector + 1;

        uint32_t startpage = page;
        uint32_t endpage = startpage + ((length+offset-
1)/256);
        uint32_t numPage = endpage - startpage + 1;

        for(int i = 0; i<numSectors; i++){
                W25Q_Sector_Erase(startsector++);
        }

        uint32_t dataPosition = 0;

        for(int i = 0; i<numPage; i++){
                uint32_t memAddr = (startpage*256) +
offset;
                uint16_t byteremaining =
bytetowrite(length, offset);
                write_enable();
                tData[0] = 0x02;
                tData[1] = (memAddr >> 16) & 0xFF;
                tData[2] = (memAddr >> 8) & 0xFF;
                tData[3] = (memAddr) & 0xFF;
                uint16_t bytetosend = byteremaining +
indx;
                for(int i =0; i < byteremaining; i++){
                        tData[indx++] =
data[i+dataPosition];
                }
                csLOW();
                SPI_Write(tData, bytetosend);
                csHIGH();
                startpage++;
                offset = 0;
                length = length - byteremaining;
                dataPosition = dataPosition+byteremaining;
                W25Q_Delay(10);
                write_disable();
        }
}

int main(){
               sprintf (TxData, "Hello from W25Q");
               W25Q_Write(0, 0, strlen(TxData), TxData);
               W25Q_FastRead(0, 0, 600, RxData);
                                              // Use
W25Q_FastRead() function, to check W25Q_Write() function
works properly or not.
}
```

The W25Q_Write_Page function takes the following parameters

- **@page** is the start page, where the write will start from.
- **@offset** is the offset on the first page. This can vary between 0 to 255.
- **@length** is the size of data we want to write.
- **@data** is the pointer to the data that we want to write.

Expression	Type	Value
[0..99]		[4096]
RxData[0]	uint8_t	72 'H'
RxData[1]	uint8_t	101 'e'
RxData[2]	uint8_t	108 'l'
RxData[3]	uint8_t	108 'l'
RxData[4]	uint8_t	111 'o'
RxData[5]	uint8_t	32 ' '
RxData[6]	uint8_t	102 'f'
RxData[7]	uint8_t	114 'r'
RxData[8]	uint8_t	111 'o'
RxData[9]	uint8_t	109 'm'
RxData[10]	uint8_t	32 ' '
RxData[11]	uint8_t	87 'W'
RxData[12]	uint8_t	50 '2'
RxData[13]	uint8_t	53 '5'
RxData[14]	uint8_t	81 'Q'

Data write successfully using W25Q_Write() function.

Waveform captured as respect to the W25Q_Write() function.

Hardware Support

This firmware is designed to work seamlessly with the

- W25Q32FV (32Mbit/4Mbyte) flash storage modules.
- W25Q128JV (128Mbit/16Mbyte) flash storage modules.
- W25Q64JV (64Mbit/8Mbyte) flash storage modules.

I have rigorously tested the firmware with all these modules and observed flawless performance across the board. This confirms the firmware's

robustness and compatibility, ensuring reliable operation for various embedded applications.

Optimizing and Troubleshooting

When working with flash memory, it's crucial to optimize your firmware for speed and reliability. Consider the following tips:

- **Buffer Management**: Use DMA for large data transfers to offload the CPU and reduce transfer times.
- **Power Management**: Implement power-down modes in your firmware to save power when the flash memory is not in use.
- **Error Handling**: Implement robust error checking and handling routines, especially for write and erase operations.

Conclusion

The W25Q series flash memory offers a robust solution for non-volatile data storage in embedded systems. By mastering the firmware development process, you can unlock the full potential of these memory devices, ensuring your products are reliable and efficient.

I hope this article provides you with valuable insights into W25Q flash memory and guides you in your firmware development journey.Follow for more insights on embedded system design and firmware development. Feel free to reach out if you have any questions or need guidance on similar projects!

Best Regards,

Subhrajit Majumder.

Comments

5 · 2 comments

Like Comment Share

Add a comment...

Most recent

Enjoyed this article?

Follow to never miss an update.

NOVAL BOBY ANTONY • 2nd 7mo

Attended APJ Abdul Kalam Technol[illegible]versity

Hello sir

Is it possible to interfe[illegible]

Like | Reply · 1 reply

Subhrajit Majumder

Post Graduate Embedded Developer (Research and Development) | Embedded Software | Embedded Hardware | Embedded Product Development | IoT

Subhrajit Majumder ´ Follow 7mo

Post Graduate Embedded [illegible]earch and Development...

NOVAL BOBY ANTONY

Yes, it is possible to interface W25Q SPI NOR Flash with a FAT filesystem (FATFS) on an STM32 microcontroller.

To use FATFS with W25Qxx, you need a wear-leveling flash translation lay[illegible]

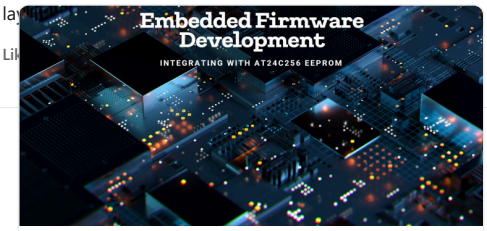
Lik[illegible]

More articles for you

Microcontroller

Ramana I

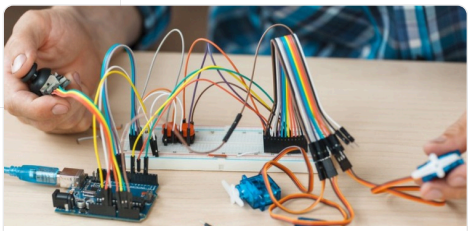
1



🚀 Firmware Basics for AT24C256 EEPROM with STM32 Integration! 💡

Subhrajit Majumder

8 · 1 comment



How to Interface an LED With an 8051 Microcontroller: Processes and Applications

PCB Manufacturing & Assembly

34 · 2 comments · 5 reposts